

Week 10: Replication, High Availability, Patroni, and PgBouncer

Phase 4: Architecture & Career | Week 10 of 12

Week Overview

This week covers the infrastructure layer that makes PostgreSQL production-safe: streaming replication, logical replication, high-availability with Patroni, and connection pooling with PgBouncer. These are the architectures that power databases at companies processing millions of requests per day.

Focus: Understand the trade-offs at each layer. Interviews for senior roles always include HA and replication questions.

Learning Objectives

By the end of this week, you will be able to:

- Configure streaming replication (primary + standby).
 - Explain synchronous vs. asynchronous replication trade-offs.
 - Set up logical replication for selective table replication.
 - Describe how Patroni manages automatic failover.
 - Configure PgBouncer in transaction pooling mode.
 - Explain what happens during a failover event.
 - Read and interpret replication lag metrics.
 - Design a multi-region database architecture.
-

Required Reading

- [13_Replication_HA/](#) — All files
-

Daily Schedule

Monday — Streaming Replication Theory and Setup (60 min)

Topics:

- WAL (Write-Ahead Log): the foundation of replication
- Streaming replication: primary streams WAL to standby in real time
- Synchronous vs. asynchronous replication
- Hot standby: read queries on replica
- Replication slots: prevent WAL deletion before standby receives it
- `pg_stat_replication` : monitoring replication health

```
-- On primary: create replication user
CREATE USER replicator WITH REPLICATION ENCRYPTED PASSWORD 'replpassword';

-- postgresql.conf on primary:
```

```
-- wal_level = replica
-- max_wal_senders = 3
-- max_replication_slots = 3
-- synchronous_standby_names = '' (async) or 'standby1' (sync)

-- pg_hba.conf on primary:
-- host replication replicator <standby_ip>/32 scram-sha-256
```

```
# On standby: initial sync with pg_basebackup
pg_basebackup -h primary_ip -U replicator -D /var/lib/postgresql/data \
    --wal-method=stream -P -R
# -R creates recovery configuration automatically

# Start standby PostgreSQL
systemctl start postgresql

# Verify replication on primary
# SELECT * FROM pg_stat_replication;
```

```
-- Monitor replication lag
SELECT
    client_addr,
    application_name,
    state,
    sent_lsn,
    write_lsn,
    flush_lsn,
    replay_lsn,
    write_lag,
    flush_lag,
    replay_lag
FROM pg_stat_replication;

-- Check if standby is falling behind
SELECT now() - pg_last_xact_replay_timestamp() AS replication_lag
FROM pg_stat_replication; -- Run on standby

-- Replication slots (prevent WAL cleanup)
SELECT slot_name, slot_type, active, restart_lsn, wal_status
FROM pg_replication_slots;
```

Tuesday — Logical Replication (90 min)

Topics:

- Logical vs. physical replication
- Logical replication use cases: selective table sync, zero-downtime migration, CDC
- Publication and subscription model
- Conflicts in logical replication

- Limitations: sequences, DDL, large objects

```
-- On publisher (primary):
-- postgresql.conf: wal_level = logical

-- Create publication (all tables or specific)
CREATE PUBLICATION my_pub FOR ALL TABLES;
-- Or selective:
CREATE PUBLICATION orders_pub FOR TABLE orders, order_items;

-- On subscriber (replica or different database):
CREATE SUBSCRIPTION my_sub
    CONNECTION 'host=publisher_ip dbname=mydb user=replicator password=replpassword'
    PUBLICATION my_pub;

-- Check subscription status
SELECT subname, subenabled, subslotname
FROM pg_subscription;

SELECT pid, relid::regclass, received_lsn, latest_end_lsn, latest_end_time
FROM pg_stat_subscription;

-- Useful patterns:
-- 1. Copy table to new database for testing (no downtime)
-- 2. Replicate subset of data to analytics replica
-- 3. Replicate to different PostgreSQL version for migration

-- Stop and drop subscription
ALTER SUBSCRIPTION my_sub DISABLE;
DROP SUBSCRIPTION my_sub;
```

Wednesday — High Availability with Patroni (90 min)

Topics:

- The HA problem: automatic failover without split-brain
- How Patroni works: DCS (etcd/Consul/ZooKeeper) for leader election
- Patroni configuration file
- Failover scenarios: planned switchover vs. emergency failover
- patronictl commands
- HAProxy for read/write routing
- The difference between Patroni, repmgr, and pg_auto_failover

```
# /etc/patroni/patroni.yml (example)
scope: my_cluster
namespace: /db/
name: pg-node-1

restapi:
  listen: 0.0.0.0:8008
```

```

connect_address: 192.168.1.10:8008

etcd:
  hosts: etcd1:2379,etcd2:2379,etcd3:2379

bootstrap:
  dcs:
    ttl: 30
    loop_wait: 10
    retry_timeout: 10
    maximum_lag_on_failover: 1048576 # 1MB max lag for failover candidate
  initdb:
    - encoding: UTF8
    - data-checksums

postgresql:
  listen: 0.0.0.0:5432
  connect_address: 192.168.1.10:5432
  data_dir: /var/lib/postgresql/data
  authentication:
    replication:
      username: replicator
      password: replpassword
    superuser:
      username: postgres
      password: superpassword
  parameters:
    max_connections: 200
    shared_buffers: 2GB
    wal_level: logical
    max_replication_slots: 10
    max_wal_senders: 10

```

```

# Patroni CLI commands
patronictl -c /etc/patroni/patroni.yml list           # View cluster state
patronictl -c /etc/patroni/patroni.yml topology      # Show topology
patronictl -c /etc/patroni/patroni.yml switchover    # Planned primary switch
patronictl -c /etc/patroni/patroni.yml failover      # Emergency failover
patronictl -c /etc/patroni/patroni.yml reinit pg-node-2 # Reinitialize a node
patronictl -c /etc/patroni/patroni.yml pause         # Pause automatic failover
patronictl -c /etc/patroni/patroni.yml resume        # Resume automatic failover

```

Thursday — PgBouncer Connection Pooling (60 min)

Topics:

- Why connection pooling: PostgreSQL connection overhead
- Session vs. Transaction vs. Statement pooling modes
- PgBouncer configuration
- Monitoring pool utilization

- PgBouncer limitations (SET, LISTEN, prepared statements in transaction mode)

```
# /etc/pgbouncer/pgbouncer.ini

[databases]
; Application database
mydb = host=127.0.0.1 port=5432 dbname=mydb

; For Patroni/HA: point to HAProxy VIP
mydb_ha = host=haproxy-vip port=5432 dbname=mydb

[pgbouncer]
; Recommended for web apps: transaction pooling
pool_mode = transaction

listen_addr = 0.0.0.0
listen_port = 6432

auth_type = scram-sha-256
auth_file = /etc/pgbouncer/userlist.txt

; Pool sizing
max_client_conn = 5000      ; Clients can hold 5000 connections to pgbouncer
default_pool_size = 25      ; But only 25 backend connections per database/user
min_pool_size = 5           ; Always keep 5 connections warm
reserve_pool_size = 5       ; Extra for burst

; Timeouts
client_idle_timeout = 600
server_idle_timeout = 600
server_connect_timeout = 15

; Monitoring
stats_period = 60
log_connections = 1
log_disconnections = 1
```

```
-- PgBouncer admin console (psql -h localhost -p 6432 -U pgbouncer pgbouncer)
SHOW POOLS;
-- cl_active: active client connections
-- sv_active: active server connections
-- sv_idle: idle server connections

SHOW STATS;
-- total_requests, avg_req, avg_recv, avg_sent

SHOW CLIENTS;
SHOW SERVERS;
RELOAD;    -- Apply config changes
```

Friday — HA Architecture Design + Review (45 min)

Architecture Design Exercise:

Design a 3-node PostgreSQL HA cluster for an e-commerce platform:

Requirements:

- 10,000 read queries/sec, 1,000 write queries/sec
- 99.99% uptime (< 1 hour downtime/year)
- RPO: 0 (no data loss)
- RTO: < 30 seconds failover
- 5TB data, growing 100GB/month

Draw the architecture including:

- Primary and replica placement
- PgBouncer placement
- Patroni + etcd quorum nodes
- HAProxy routing rules
- Backup strategy
- Monitoring stack

Key decisions to justify:

1. Synchronous or asynchronous replication? Why?
2. How many replicas? What roles (read replicas, standby)?
3. How do you route reads vs. writes?
4. How do you handle the failover window for in-flight transactions?
5. Where do backups go and how frequently?

Practice Tasks

1. Set up streaming replication between two local PostgreSQL instances (use Docker).
2. Verify replication lag using `pg_stat_replication`.
3. Simulate a primary failure and promote the standby manually.
4. Create a logical replication publication for specific tables.
5. Configure PgBouncer in transaction mode and verify connections are pooled.
6. Explain what happens to in-flight transactions during a Patroni failover.
7. Draw the full architecture for a 3-node Patroni cluster with PgBouncer.
8. Write a monitoring query that alerts when replication lag exceeds 60 seconds.

Self-Assessment Checklist

- ☐ I can explain streaming replication in my own words
 - ☐ I understand the difference between sync and async replication
 - ☐ I know how Patroni prevents split-brain scenarios
 - ☐ I can explain the three PgBouncer pool modes
 - ☐ I drew the 3-node HA architecture with all components
 - ☐ I understand what RPO and RTO mean and how to achieve them
 - ☐ I can monitor replication health with `pg_stat_replication`
-

Mock Interview Questions

1. Explain streaming replication. What is WAL and how does it flow to replicas?
 2. What is the difference between synchronous and asynchronous replication?
 3. How does Patroni prevent split-brain in a 3-node cluster?
 4. What is a replication slot and why can it be dangerous?
 5. Explain the three PgBouncer pool modes. Which would you use for a REST API?
 6. Your primary PostgreSQL goes down. Walk me through what Patroni does.
 7. What is the difference between logical and physical replication?
 8. How do you achieve zero-downtime migrations with logical replication?
 9. What happens to in-flight transactions when a failover occurs?
 10. Design a database architecture for 99.99% uptime. What are the components?
-

Resources

- This repo: `13_Replication_HA/`
- Patroni docs: <https://patroni.readthedocs.io>
- PgBouncer docs: <https://www.pgбouncer.org>
- Crunchy Data HA tutorial: <https://access.crunchydata.com>